Experiment 10: Data Synopsis Generation using Bloom Filter and Count-Min Sketch

Aim

To implement two popular data synopsis techniques, the Bloom filter and the Count-Min Sketch, for efficiently representing and querying a data stream while using very small memory.

Objectives

- Implement a Bloom filter.
- Implement a Count-Min Sketch.
- Compare the exact and approximate counts.

Theory

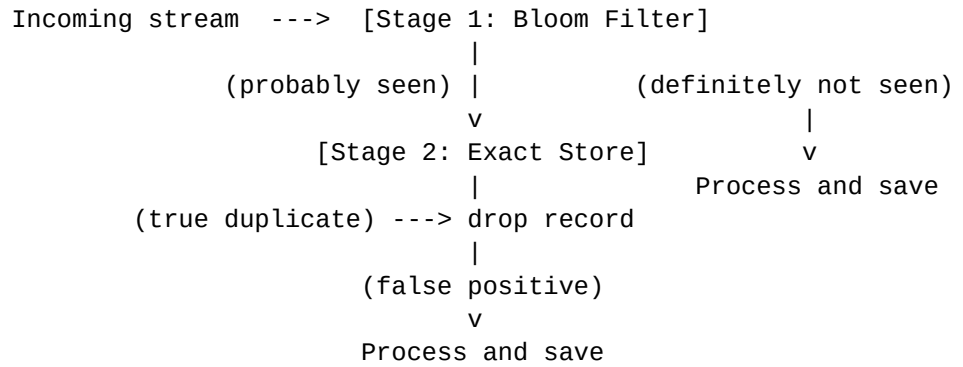
A data synopsis is a small summary structure that represents a very large data stream approximately, using far less memory than storing the stream itself. Two of the most widely used synopsis structures are the Bloom filter and the Count-Min Sketch.

Bloom filter. It answers membership questions. It uses a bit array and several hash functions. When an item is inserted, every hash position is set to 1. When an item is queried, if any of its hash positions is 0 the item is definitely not present. If all positions are 1 the item is probably present, because those bits may have been set by other items. A Bloom filter can therefore produce false positives but never false negatives.

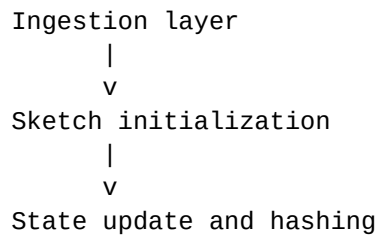
Count-Min Sketch. It answers frequency questions. It uses a two dimensional table of counters with one hash function per row. When an item arrives, one counter in each row is increased. The estimated count is the minimum of those counters, because the minimum is the least affected by collisions. The estimate is therefore never lower than the true count.

Data Pipeline

Bloom Filter



Count-Min Sketch



```
       |
       v
Query and aggregation (Count-Min Sketch)
```

Requirements

- Python 3.x
- Jupyter Notebook
- Hashlib module

Step 1: Library Relation

```
import hashlib
import random
```

Interpretation: hashlib provides the MD5 hash function used to generate the hash positions.

Step 2: Define the Bloom Filter

```
class BloomFilter:

    def __init__(self, size, hash_count):
        self.size = size
        self.hash_count = hash_count
        self.bit_array = [0] * size
```

Interpretation: Creates a bit array of the given size with all bits set to 0, and stores the number of hash functions to be used.

Step 3: Generate Multiple Hash Values

```
    def get_hashes(self, item):

        hashes = []

        for i in range(self.hash_count):
            data = item + str(i)
            hash_value = int(hashlib.md5(data.encode()).hexdigest(), 16)
            hashes.append(hash_value % self.size)

        return hashes
```

Interpretation: A different string is created for each hash function by appending the loop counter to the item. The MD5 digest is converted to an integer and reduced modulo the array size to give a valid bit position.

Step 4: Set the Corresponding Bits

```
    def insert(self, item):

        indexes = self.get_hashes(item)

        for index in indexes:
            self.bit_array[index] = 1
```

Interpretation: Sets every bit position returned by the hash functions to 1.

Step 5: Return False if Any Bit is Zero

```
    def check(self, item):
```

```
        indexes = self.get_hashes(item)

        for index in indexes:
            if self.bit_array[index] == 0:
                return False

        return True
```

Interpretation: If any bit is 0 the item was definitely never inserted. If all bits are 1 the item is probably present.

Step 6: Stream Generation

```
stream = [
    "A", "B", "A", "C", "A", "B", "D", "A", "E", "A",
    "C", "B", "F", "A", "B", "A", "G", "A", "H", "A"
]

print("Stream")
print(stream)
```

Interpretation: Defines the incoming data stream of 20 elements with 8 distinct values.

Step 7: Bloom Filter

```
print("\nBloom Filter")

bf = BloomFilter(size=50, hash_count=3)

for item in stream:
    bf.insert(item)

queries = ["A", "B", "Z", "H"]

for q in queries:
    if bf.check(q):
        print(q, "-> probably present")
    else:
        print(q, "-> definitely not present")
```

Interpretation: Builds a Bloom filter of 50 bits with 3 hash functions, inserts every stream element and then queries four elements. Z was never part of the stream.

Bloom Filter Inference

- The Bloom filter performs fast and memory efficient membership testing.
- The algorithm guarantees no false negatives, meaning that if an element is reported as "definitely not present" it is truly absent.
- An element reported as a false positive is only "probably present".

Step 8: Define the Count-Min Sketch

```
class CountMinSketch:

    def __init__(self, width, depth):
        self.width = width
        self.depth = depth
        self.table = [[0] * width for i in range(depth)]
```

```
    def get_hashes(self, item):

        hashes = []

        for i in range(self.depth):
            data = item + str(i)
            hash_value = int(hashlib.md5(data.encode()).hexdigest(), 16)
            hashes.append(hash_value % self.width)

        return hashes

    def insert(self, item):

        indexes = self.get_hashes(item)

        for row in range(self.depth):
            self.table[row][indexes[row]] += 1

    def estimate(self, item):

        indexes = self.get_hashes(item)

        counts = []

        for row in range(self.depth):
            counts.append(self.table[row][indexes[row]])

        return min(counts)
```

Interpretation: Creates a table of depth rows and width columns, all set to 0. Each row has its own hash function. Inserting an item increases one counter in every row, and the estimated count is the minimum of those counters.

Step 9: Count-Min Sketch

```
print("\nCount-Min Sketch")

cms = CountMinSketch(width=20, depth=3)

for item in stream:
    cms.insert(item)
```

Interpretation: Builds a sketch of 3 x 20 counters and feeds the whole stream into it.

Step 10: Compare the Exact and Approximate Counts

```
exact_count = {}

for item in stream:
    if item in exact_count:
        exact_count[item] += 1
    else:
        exact_count[item] = 1

print("Element   Exact   Approx   Error")

for element in sorted(exact_count):
    exact = exact_count[element]
```

```
    approx = cms.estimate(element)
    print(f"{element:^9} {exact:^7} {approx:^8} {approx - exact:^5}")
```

Interpretation: The exact count is calculated with a dictionary and compared against the sketch estimate. The error is always zero or positive, because the sketch never underestimates.

Sample Input

```
stream = ["A", "B", "A", "C", "A", "B", "D", "A", "E", "A",
          "C", "B", "F", "A", "B", "A", "G", "A", "H", "A"]

Bloom filter     : size = 50, hash_count = 3
Count-Min Sketch : width = 20, depth = 3
Queries          : ["A", "B", "Z", "H"]
```

Expected Output

```
Stream
['A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'E', 'A',
 'C', 'B', 'F', 'A', 'B', 'A', 'G', 'A', 'H', 'A']

Bloom Filter
A -> probably present
B -> probably present
Z -> definitely not present
H -> probably present

Count-Min Sketch
Element   Exact   Approx   Error
    A        9       9       0
    B        4       4       0
    C        2       2       0
    D        1       1       0
    E        1       1       0
    F        1       1       0
    G        1       1       0
    H        1       1       0
```

Result

Element	Output
A	Probably present
B	Probably present
Z	Definitely not present
H	Probably present

Count-Min Sketch Inference

- The exact count is stored in a dictionary that grows with the number of distinct elements.
- The Count-Min Sketch uses a fixed table of 3 x 20 counters no matter how many distinct elements arrive.
- The estimated count is never lower than the exact count, so the error is always zero or positive.
- With this stream size and table size there are no collisions, so the estimate matches the exact count for every element.

Conclusion

- The Bloom filter was successfully implemented using Python.
- It efficiently performed membership testing with very low memory usage.
- The experiment showed that a Bloom filter can produce false positives but never false negatives.
- The Count-Min Sketch estimated the frequency of every element using a fixed size table, and the approximate counts matched the exact counts for this stream.